

Multi-Tenancy Design

Multiple Clients on One Environment — Shared & Isolated Database Strategies

What is Multi-Tenancy?

Multi-tenancy is an architecture where a single application instance serves multiple tenants (clients/organizations), each with isolated data and configuration. The goal is cost efficiency (shared infrastructure) while ensuring data isolation, security, and customizability per tenant.

Term	Definition
Tenant	An isolated organizational unit — typically a company, team, or customer — that uses the application
Tenant ID	A unique identifier (UUID, slug) used to isolate data: every row, object, and resource belongs to exactly one tenant
Data Isolation	Guarantee that Tenant A cannot read, write, or even infer the existence of Tenant B's data
Shared infrastructure	Same application servers, cache, and optionally same database serve all tenants — reduces cost
Customization	Tenants may need different configs: feature flags, branding, locale, rate limits, SLA tiers

Database Isolation Strategies

Strategy 1: Separate Database per Tenant (Full Isolation)

```
Tenant A -> Database A (tenant_a_db) Tenant B -> Database B (tenant_b_db) Tenant C -> Database C (tenant_c_db)
// Spring Boot: dynamic DataSource routing @Component public class TenantDataSourceRouter extends
AbstractRoutingDataSource { @Override protected Object determineCurrentLookupKey() { return
TenantContext.getCurrentTenant(); // thread-local tenant ID } }
```

Attribute	Detail
Isolation	Maximum — separate DB process, no cross-tenant query possible
Customization	Schema per tenant: different columns, indexes, even DB engines
Compliance	Easiest for GDPR/HIPAA: delete tenant = drop database
Scaling	Each DB scales independently; no noisy-neighbor problem
Cons	Expensive: N DBs = N connection pools, N backup jobs, N DB licenses Operational overhead: schema migrations must run on all N databases
Best For	Enterprise B2B SaaS: healthcare, finance, legal. Tenants with strict compliance or large data volumes (e.g. Salesforce Hyperforce)

Strategy 2: Separate Schema per Tenant (Same DB, Different Schema)

```
-- PostgreSQL schemas CREATE SCHEMA tenant_abc; -- Tenant A CREATE SCHEMA tenant_xyz; -- Tenant B -- Each
schema has same tables: orders, users, products etc. -- BUT data is physically separated -- Hibernate
multi-tenancy: SCHEMA mode spring.jpa.properties.hibernate.multiTenancy=SCHEMA
spring.jpa.properties.hibernate.tenant_identifier_resolver=com.app.TenantIdentifierResolver
spring.jpa.properties.hibernate.multi_tenant_connection_provider=com.app.TenantConnectionProvider
```

Attribute	Detail
Isolation	Strong — schemas are logical boundaries within one DB server
Overhead	Moderate — one DB process, but many schemas. ~1000 schema limit in practice
Schema Migration	Run Flyway/Liquibase against each schema on deploy. Parallelizable but slow for many tenants
Cons	Cross-schema queries possible (SQL: SELECT * FROM tenant_abc.orders) — rely on app-layer control Connection pool shared; one schema crash can affect others
Best For	Mid-market SaaS with moderate tenant count (<500). Good balance of isolation vs cost

Strategy 3: Shared Schema — Row-Level Tenant ID (Shared DB, Shared Tables)

```
-- Every table has tenant_id column CREATE TABLE orders ( id BIGSERIAL PRIMARY KEY, tenant_id UUID NOT NULL,
-- *** always filter by this *** customer_id BIGINT, amount DECIMAL, created_at TIMESTAMPTZ DEFAULT NOW() );
CREATE INDEX idx_orders_tenant ON orders(tenant_id, created_at DESC); -- Hibernate: filter @FilterDef(name = "
tenantFilter", parameters = @ParamDef(name = "tenantId", type = String.class)) @Filter(name = "tenantFilter",
condition = "tenant_id = :tenantId") @Entity public class Order { ... } // Enable filter in service
session.enableFilter("tenantFilter").setParameter("tenantId", TenantContext.getCurrentTenant());
```

Attribute	Detail
Isolation	Weak — logical only. Bugs or missing filter = data breach. Enforce at multiple layers
Cost	Lowest — one database for all tenants. Ideal for SMB/startup
Scaling	DB is shared noisy neighbor. One large tenant can degrade all others
Migration	Simple — one schema to migrate for all tenants
Compliance	GDPR deletion: WHERE tenant_id = :id. But multi-tenant DB means shared audit trails
Best For	Consumer SaaS, SMB multi-tenancy, project management tools, high tenant count (>10k tenants)

Strategy Comparison

Criteria	Separate DB	Separate Schema	Shared Table (Row)
Data Isolation	Maximum	Strong	Logical only
Cost per Tenant	High	Medium	Low
Max Tenants	Hundreds	~500-1000	Millions
Schema Migration	Complex — per DB	Moderate — per schema	Simple — once
Noisy Neighbor	None	DB-level only	Table-level risk
Compliance / GDPR	Easiest	Good	Requires care
Cross-tenant query risk	None	Low	High — must enforce
Typical example	Salesforce, enterprise ERP	GitLab, mid-market	Slack, Notion, Asana

Hybrid Strategy: Tiered Isolation

Many real-world SaaS apps offer different isolation levels per pricing tier:

Tier	DB Strategy	Target Customer
Free / Starter	Shared DB + shared schema (row-level tenant_id)	Individual users, SMBs
Business	Shared DB + dedicated schema per tenant	Mid-size companies
Enterprise	Dedicated database instance per tenant	Large enterprises, regulated industries

The application uses a tenant metadata table to look up which isolation strategy to use for a given tenant, then routes accordingly.

Tenant Context Propagation (Spring Boot)

TenantContext — Thread-Local Storage

```
public class TenantContext { private static final ThreadLocal<String> CURRENT_TENANT = new ThreadLocal<>();
public static void setCurrentTenant(String tenantId) { CURRENT_TENANT.set(tenantId); } public static String
getCurrentTenant() { return CURRENT_TENANT.get(); } public static void clear() { CURRENT_TENANT.remove(); } //
ALWAYS clear after request }
```

Extract Tenant from JWT / Header

```
@Component public class TenantFilter extends OncePerRequestFilter { @Override protected void
doFilterInternal(HttpServletRequest req, HttpServletResponse res, FilterChain chain) throws ServletException,
IOException { // Option 1: Extract from JWT claim String token = extractBearerToken(req); String tenantId =
jwtService.extractClaim(token, "tenantId"); // Option 2: Extract from subdomain: tenant.myapp.com // String
host = req.getServerName(); // "acme.myapp.com" // String tenantId = host.split("\\.")[0]; // "acme" // Option
```

```
3: From custom header X-Tenant-ID // String tenantId = req.getHeader("X-Tenant-ID"); try
{TenantContext.setCurrentTenant(tenantId); chain.doFilter(req, res); } finally { TenantContext.clear();
//CRITICAL: prevent leak to next request } } }
```

Propagate to Async / Kafka Consumers

```
// For async methods, ThreadLocal doesn't propagate – use MDC or custom TaskDecorator @Bean public
TaskExecutor taskExecutor() { ThreadPoolTaskExecutor executor = new ThreadPoolTaskExecutor();
executor.setTaskDecorator(runnable -> { String tenantId = TenantContext.getCurrentTenant(); return () -> {
TenantContext.setCurrentTenant(tenantId); try { runnable.run(); } finally { TenantContext.clear(); } }; });
return executor; } // Kafka: include tenantId in message headers; consumer reads header and sets context
```

Row-Level Security (PostgreSQL RLS)

For shared-schema strategy, PostgreSQL RLS enforces isolation at the database engine level — even if app code forgets the WHERE clause.

```
-- Enable RLS on table ALTER TABLE orders ENABLE ROW LEVEL SECURITY; -- Create policy: each user can only see
their tenant's rows CREATE POLICY tenant_isolation ON orders USING (tenant_id =
current_setting('app.tenant_id')::uuid); -- App sets tenant before each query SET app.tenant_id = '
uuid-of-tenant'; -- or SET LOCAL in transaction SELECT * FROM orders; -- automatically filters by policy -- Or
via Spring Boot connection property: // entityManager.createNativeQuery("SET app.tenant_id = '" + tenantId + "
").executeUpdate();
```

Non-DB Concerns in Multi-Tenancy

Concern	Approach
Feature Flags per Tenant	Store feature toggles in tenant config table or Redis. e.g., <code>featureService.isEnabled("new-ui", tenantId)</code>
Rate Limiting per Tenant	Rate limit bucket keyed by <code>tenant_id</code> (Redis token bucket). Enterprise tenants get higher limits.
File Storage	S3 prefix per tenant: <code>s3://bucket/tenant-{id}/uploads/</code> or separate S3 buckets for enterprise
Caching	Cache keys must include <code>tenantId</code> : <code>"product:{tenantId}:{productId}"</code> to prevent cross-tenant cache hits
Logging / Monitoring	Tag all logs and metrics with <code>tenantId</code> . <code>MDC.put("tenantId", id)</code> . Grafana dashboards filterable by tenant.
Email / Notifications	Sender config (from address, SMTP) per tenant. Store in tenant config.
Background Jobs	Job must restore tenant context. Include <code>tenantId</code> in job payload/message headers.
API Rate Response	Include tenant tier in error: <code>{"error":"Rate limit exceeded","tier":"STARTER","limit":100}</code>

Interview Q&A;

Q1: What are the three main multi-tenancy database strategies?

(1) Separate database per tenant — maximum isolation, highest cost, best for enterprise/compliance; (2) Separate schema per tenant — same DB server, different schemas, good isolation at moderate cost; (3) Shared schema with `tenant_id` column — lowest cost, logical isolation only, risk of data leakage if not carefully enforced. Many SaaS apps offer all three as pricing tiers.

Q2: How do you prevent cross-tenant data leakage in a shared schema?

Defense in depth: (1) Add `tenant_id` to every table and create composite indexes (`tenant_id, other_cols`); (2) Enable Hibernate `@Filter` that automatically appends `WHERE tenant_id = :tenantId`; (3) Enable PostgreSQL Row-Level Security as a DB-enforced backstop; (4) Centralize data access in repositories — no raw JDBC queries scattered in services; (5) Integration tests that verify Tenant A cannot access Tenant B data.

Q3: Where do you store the tenant ID for each request?

Best practice: encode `tenantId` as a JWT claim (`"tenantId": "uuid"`) in the access token. A Spring filter decodes the JWT and sets `TenantContext.setCurrentTenant(tenantId)` in a `ThreadLocal` at the start of each request, and clears it in a finally block. Alternatives: subdomain routing (`acme.app.com` → `tenantId="acme"`), X-Tenant-ID header (internal microservices), or session attribute.

Q4: How do you run database migrations in a multi-tenant setup?

For separate DB/schema strategies: maintain a tenant registry table. On deploy, run a Flyway migration for each tenant's `DataSource` in parallel (use an executor service). For shared schema: single Flyway migration updates the shared schema — all tenants benefit instantly. Consider a migration coordinator service that tracks per-tenant migration state and retries failed tenant migrations without blocking others.

Q5: What is the noisy-neighbor problem and how do you solve it in a shared-schema setup?

A single large tenant generates heavy queries that degrade DB performance for all other tenants. Solutions: (1) Per-tenant rate limiting at the application layer; (2) Per-tenant connection pool limits; (3) Read replicas — route heavy tenants to dedicated replicas; (4) Move large/enterprise tenants to separate DB (tier upgrade); (5) Query timeouts per tenant; (6) Priority queues for background jobs so one tenant's bulk export doesn't starve others.

Q6: How do you handle tenant onboarding and offboarding?

Onboarding: create tenant record in metadata table, provision schema/database if needed, run schema migration, seed default config, create admin user. Use a TenantProvisioningService with idempotent operations. Offboarding/GDPR deletion: mark tenant as deleted, export data if requested, delete all rows WHERE tenant_id = :id (shared schema), or DROP SCHEMA/DATABASE (isolated). Use soft-delete first, then schedule hard delete after retention period.

Q7: How do you propagate tenant context to async threads and Kafka consumers?

ThreadLocal does not propagate to child threads. Solutions: (1) Custom TaskDecorator for Spring's ThreadPoolTaskExecutor — capture tenantId at submit time, restore it in the runnable; (2) For Kafka consumers: include tenantId in Kafka message headers; consumer extracts header and sets TenantContext before processing; (3) Use MDC (Mapped Diagnostic Context) for logging, which also requires propagation via MDC.getCopyOfContextMap().

Q8: How do you cache data safely in a multi-tenant application?

Every cache key must include the tenantId: "product:{tenantId}:{productId}". Never cache without the tenant dimension — a shared key would let Tenant B read Tenant A's data from cache. Use separate Redis keyspaces or key prefixes per tenant. For per-tenant config/feature flags, cache with short TTL and evict explicitly on config change. In Redis Cluster, use hash tags: "{tenantId};product:123" ensures tenant data co-locates on one shard.